Progettazione: Architetture Software

PROGETTAZIONE

Costituisce la fase «ponte» fra la specifica e la codifica.

Si passa da: che cosa deve essere fatto a come deve essere fatto

Il risultato della progettazione si chiama **architettura (o progetto) del software**.

LIVELLO DI ASTRAZIONE/DETTAGLIO

Progetto di alto livello (o architetturale)

- scompone un sistema complesso in più sottosistemi
- identifica e specifica le parti del sistema e le loro inter-connessioni

Progettazione di dettaglio

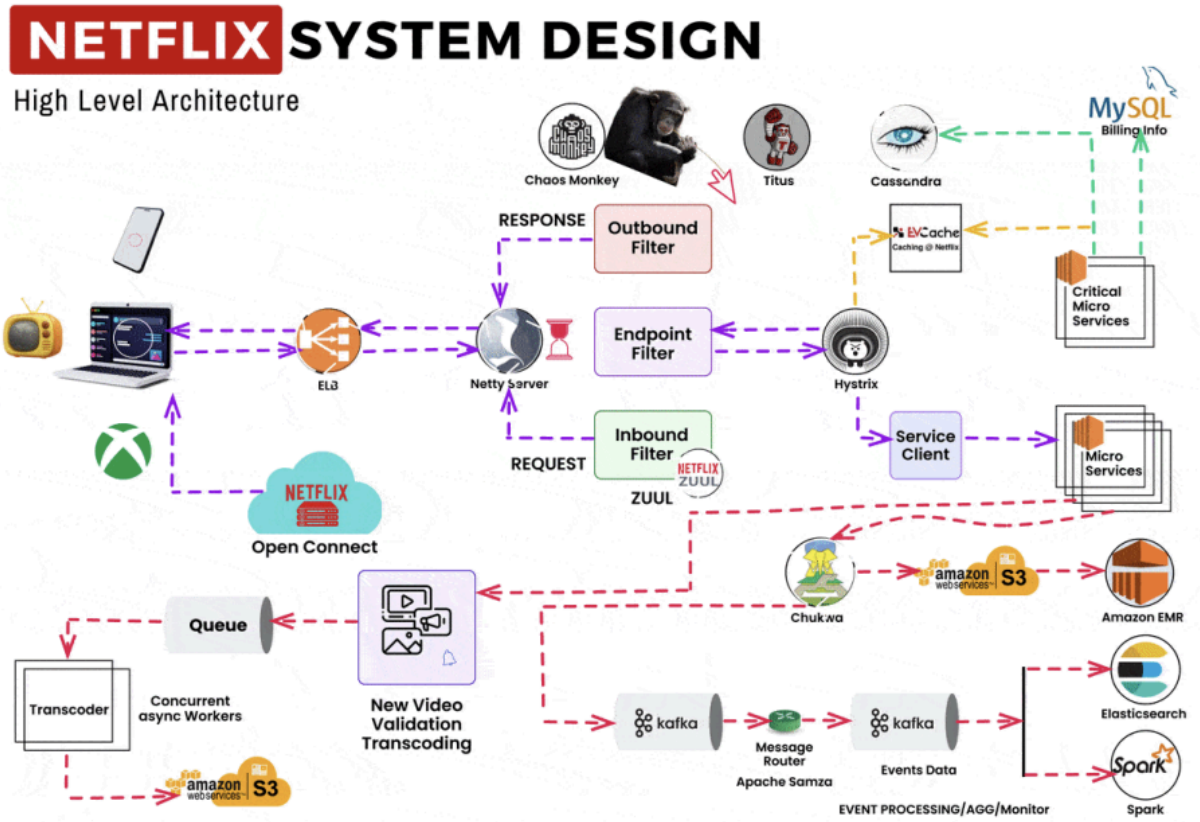
- indica come la specifica di ogni parte sarà realizzata

ARCHITETTURA SOFTWARE

In altre parole, un'architettura software:

- definisce la struttura del sistema software
- specifica le comunicazioni tra componenti
- considera aspetti funzionali e non funzionali

Fornisce un'astrazione del sistema mediante un artefatto complesso.



Tre viste interessanti (tre astrazioni interessanti)

- **Vista comportamentale**
- **Vista strutturale**
- **Vista logistica** **(di deployment / dislocazione)**

Vista comportamentale

La **vista comportamentale**, anche detta **component-and-connector (C&C)**, descrive un sistema software come **composizione di componenti software**.

Include:

- Componenti e loro interfacce

- Caratteristiche dei connettori
- Struttura del sistema in esecuzione (flusso dei dati, dinamica, parallelismo, replicazioni, ecc.)

A cosa serve:

- Consente l'analisi delle caratteristiche di qualità a tempo d'esecuzione (prestazioni, affidabilità, disponibilità, sicurezza, ecc.)
- Consente di documentare lo **stile dell'architettura**

Vista strutturale

La **vista strutturale** descrive la **struttura di un sistema software** come insieme di **unità di realizzazione o di codice** (ad esempio, classi e package).

A cosa serve:

- Analizzare le **dipendenze tra packages**
- Progettare **test di unità e di integrazione**
- Valutare la **portabilità**

Vista logistica (o di deployment)

La **vista di deployment** descrive l'**allocazione del software sugli ambienti di esecuzione**.

A cosa serve:

- Permette di valutare **prestazioni** e **affidabilità**

Approfondimento sulla vista comportamentale

Un **componente software** è un'unità di software **indipendente e riusabile**.

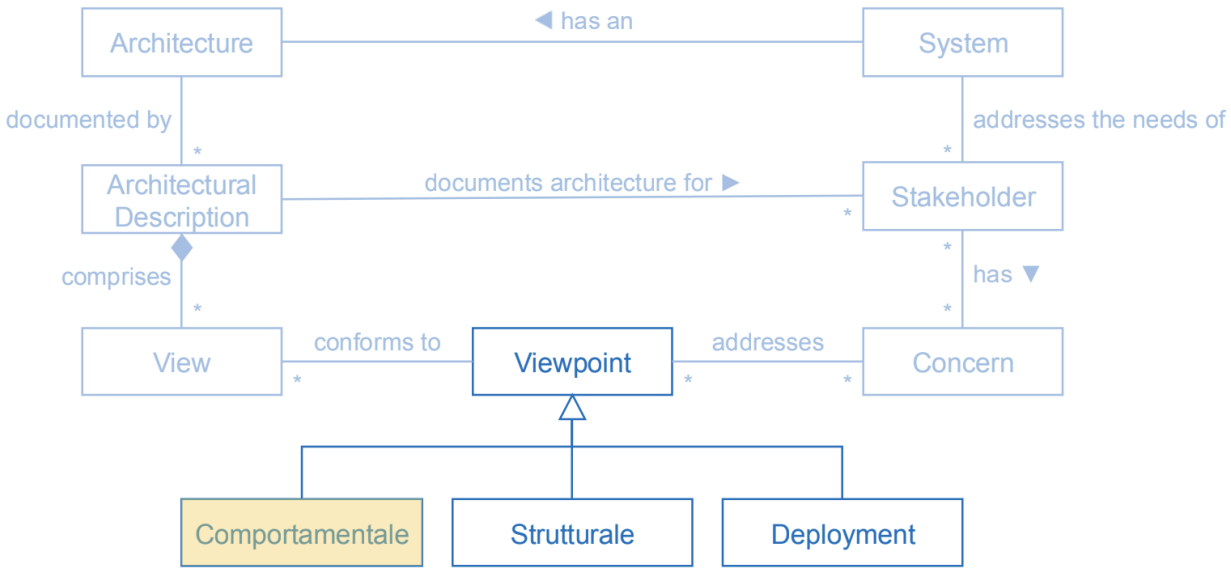
Caratteristiche:

- È un'unità concettuale di **decomposizione del sistema a tempo d'esecuzione** (es. processo, oggetto, servizio, deposito dati, ecc.)
- **Incapsula** un insieme di funzionalità e/o dati del sistema

- **Restringe l'accesso** a tali funzionalità o dati tramite **interfacce definite**
- Ha un proprio **contesto di esecuzione**
- Può essere **distribuito e installato indipendentemente** da altri componenti

Un **sistema software** è una **composizione di componenti software**, basata su:

- **Connessioni** tra più componenti
- **Interfacce** dei componenti e **connettori**



Componenti in UML

I **porti** identificano i **punti di interazione** di un componente.

Caratteristiche principali:

- Un componente può avere **più porti** (ad esempio, uno per ogni connessione)
- Un **porto** può **fornire** o **richiedere** una o più **interfacce** (omogenee)

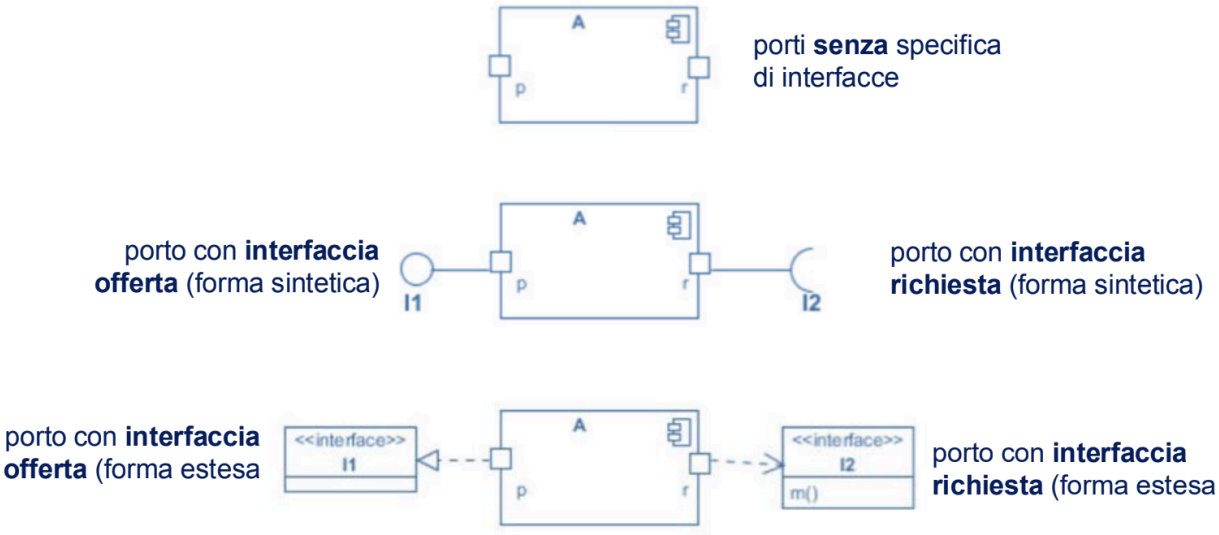
- I porti possono avere un **nome** e/o una **molteplicità** (con la sintassi usuale `1..n`)





Tre rappresentazioni equivalenti
(la terza è ridondante, ma è quella usata da Visual Paradigm)

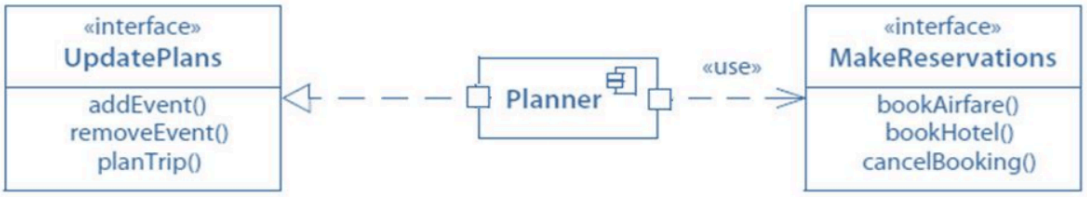
Porti e Interfacce

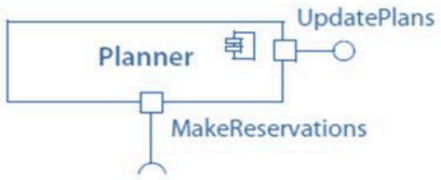


INTERFACCE: DESCRIZIONE SINTETICA VS ESTESA

Le interfacce possono essere descritte in modo

- **Sintetico**: con lollipop e forchette
- **Esteso**: per mostrare le operazioni richieste/offerte





Connettori in UML

I **connettori** sono **canali di interazione** che collegano i **porti dei componenti**, ad esempio per rappresentare:

- Protocolli
- Flussi di informazione
- Accessi ai depositi dati

In UML

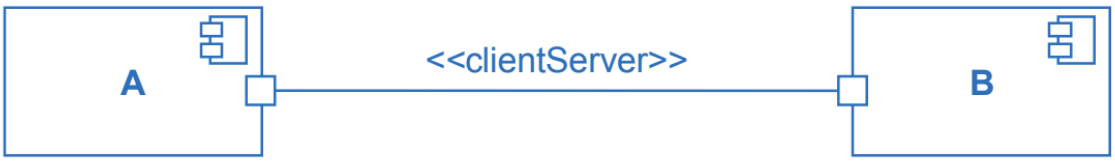
- **Non** hanno un descrittore specifico
- Si **rappresentano come associazioni**

Aggiunta di informazioni sull'interazione

È possibile indicare lo **stile della connessione** utilizzando uno **stereotipo**, ad esempio:

- `<<clientServer>>`

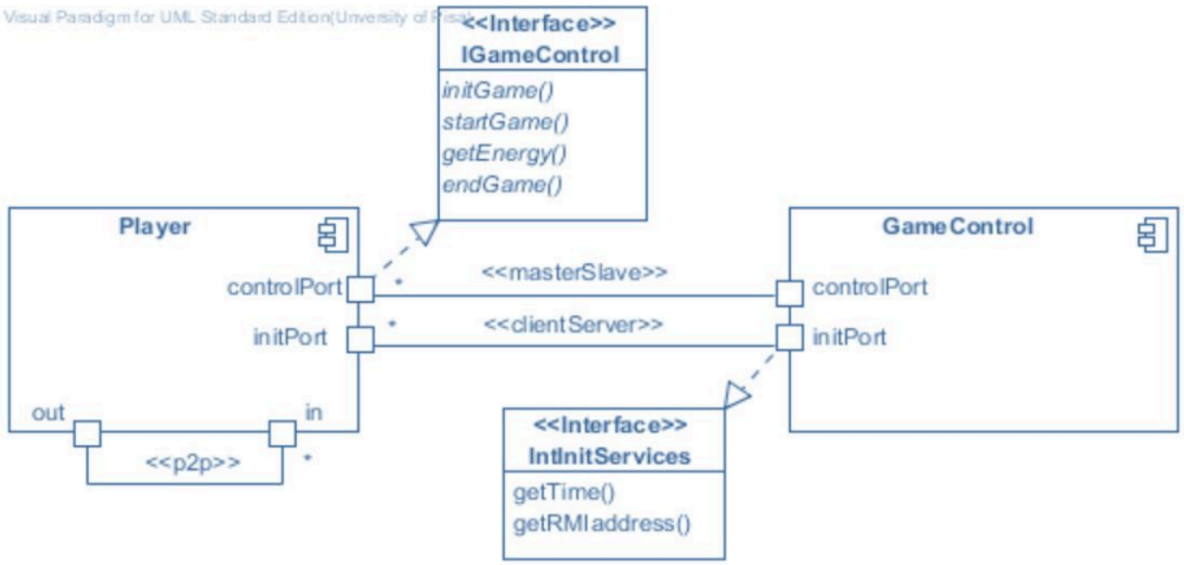
- `<<dataAccess>>` (specializzazione di `<<clientServer>>`)
- `<<pipe>>`
- `<<peer2peer>>` o `<<p2p>>`
- `<<publishSubscribe>>`



Oppure indicando i ruoli delle componenti



Esempio:



Stili (o schemi) architetturali

Uno **stile architetturale** caratterizza una **famiglia di architetture** con **caratteristiche comuni**.

Esempi:

- **Client-server** → caratterizzato dal tipo di interazione tra i componenti
- **Microservizi** → architettura basata su componenti indipendenti e comunicazione via API

Le **funzionalità** e le **interazioni tra componenti** seguono spesso **stili (schemi) standard**.

Nella vista C&C (Component-and-Connector)

Uno stile architetturale è caratterizzato da:

- **Caratteristiche generali** delle componenti coinvolte
- **Particolari interazioni** tra le componenti

  (quindi, dalle caratteristiche dei **porti** e dei **connettori**)

Stile – Pipes & Filters

Componenti

I componenti sono di tipo **filter (filtro)**:

- Trasformano uno o più **flussi di dati** dai porti di **ingresso** in uno o più flussi sui porti di **uscita**

Connettori

I connettori sono di tipo **pipe (condotta)**:

- Canale di comunicazione **unidirezionale** e **bufferizzato**
- **Preserva l'ordine** dei dati dal ruolo d'ingresso a quello d'uscita

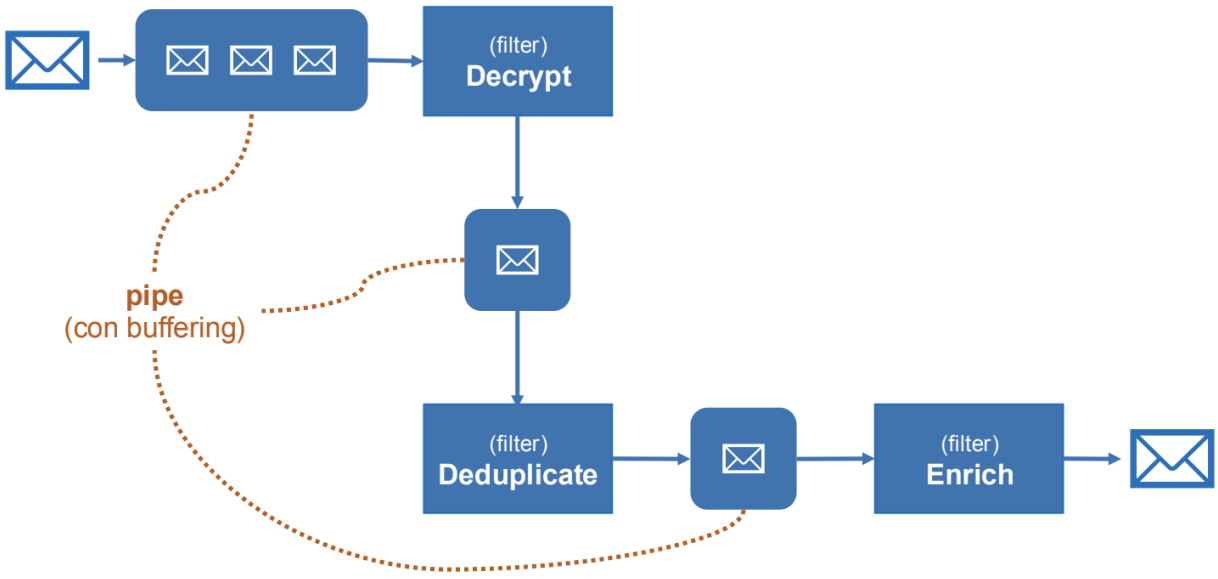
Varianti

Gli elementi dello stile possono variare nelle funzioni che svolgono, ad esempio:

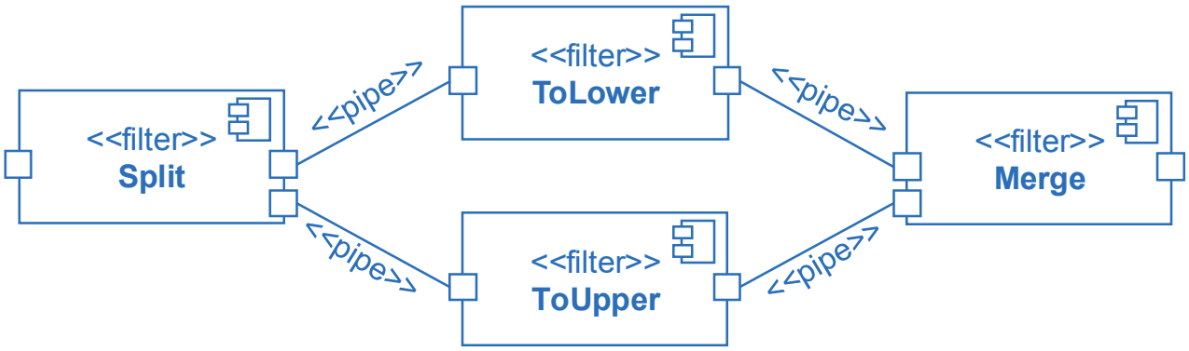
- **Pipe** con supporto per il **buffering dei dati**
- **Biforcazioni** dei flussi



Esempio con buffering



Esempio con biforcazione

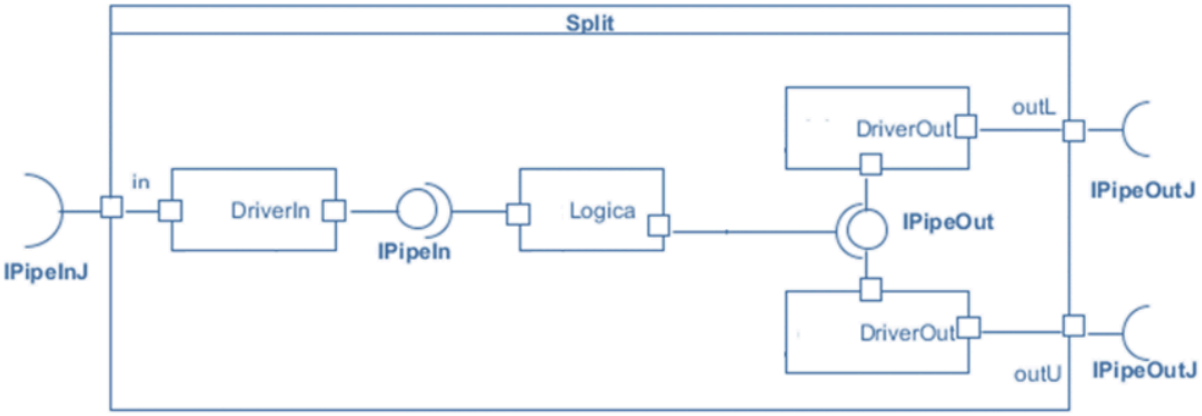


Come realizzare un filter

Ad esempio volendo realizzare questo filtro:



Possiamo realizzarlo come segue:



Stile Client-Server

Sistema formato da due componenti, il server offre un servizio

- Aspetta le richieste di un client ad un porto
- Più clienti servibili dallo stesso porto

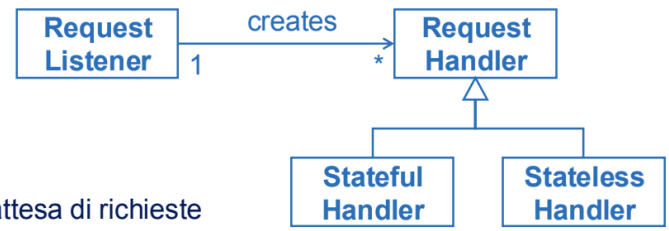
Il client invia richieste al server e attende una risposta



Come realizzare un server

Per ogni richiesta, un RequestHandler per gestirla

- Elabora la richiesta e invia la risposta al client
- Se stateless, gestisce ogni richiesta in modo indipendente
- Se stateful, consente richieste composite che consistono di più richieste atomiche
  (mantenendo un record delle richieste di un client, chiamato sessione)



Un **RequestListener** (anche multi-thread) in attesa di richieste

Stile Master-Slave

Si tratta di un caso particolare di client-server che risponde ad esigenze precise
Il servente (slave) serve un solo cliente (master)



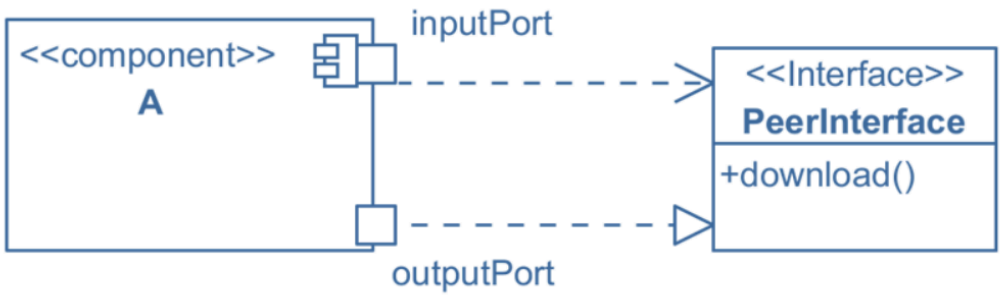
Usato, ad esempio, nella replica di database

- Database master come fonte autorevole
- Database slave come repliche sincronizzate con il master

Stile – P2P

Anche P2P (peer-to-peer) è un caso particolare di client-server

- Tutti i componenti agiscono sia da client sia da server
- Scambio di servizi alla pari
- Esempio: programmi di scambio audio-video (winmx, kazaa, eMule)



Stile – Publish-Subscribe

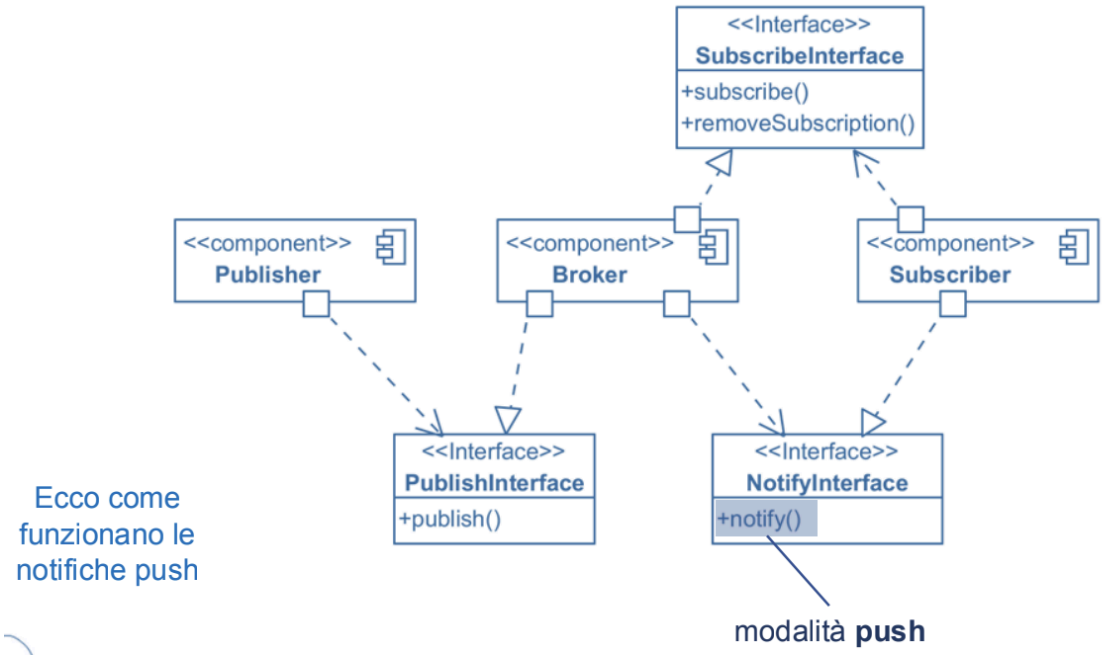
I componenti interagiscono in modo **event-based**.

Ruoli principali

- **Publisher:** produce classi di eventi
- **Subscriber:** si abbona alle classi di eventi che ritiene rilevanti
- **Broker:** smista gli eventi pubblicati

Un componente può essere **sia publisher che subscriber**, utilizzando **due connettori diversi**:

- Un connettore per le **richieste di sottoscrizione/pubblicazione**
- Un connettore per **diffondere i dati**



I **mittenti (publisher)** e i **destinatari (subscriber)** comunicano tramite un **broker**.

Funzionamento

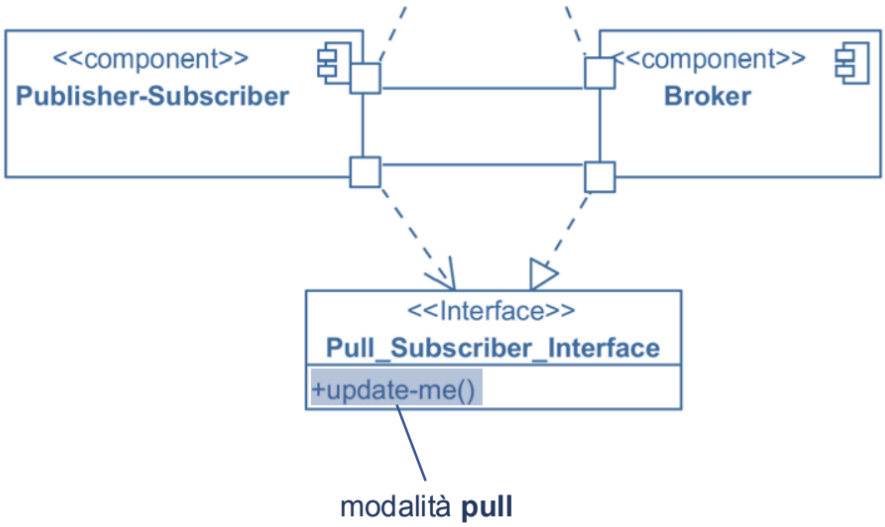
- Il **publisher** pubblica messaggi sul broker, **senza conoscere** l'identità dei destinatari
- Il **subscriber** si abbona al broker per **determinati messaggi** (ad esempio, solo quelli generati da un certo publisher o con specifiche caratteristiche)

- Il **broker** inoltra ogni messaggio ricevuto dai publisher ai **subscriber interessati**

Caratteristica principale

I **publisher** non sanno **quanti o quali subscriber** esistano (e viceversa).

→ Questo meccanismo contribuisce alla **scalabilità del sistema**.



Push o Pull?

Modello **Push**

Il **broker** invia attivamente i messaggi ai **consumatori**.

Caratteristiche:

- Il broker **controlla la frequenza** con cui i dati vengono trasferiti
- Deve decidere **quando inviare un messaggio** (immediatamente o dopo aver accumulato più dati)
- È **più complesso** gestire consumatori con esigenze e capacità diverse

Modello **Pull**

Il **consumatore** si assume la responsabilità di **recuperare i messaggi** dal broker.

Caratteristiche:

- Il consumatore deve tenere traccia del **prossimo messaggio da ricevere**
- Migliora la **scalabilità** (meno carico sul broker) e la **flessibilità** (consumatori diversi possono operare a ritmi differenti)
- Se non ci sono messaggi nel broker, i consumatori potrebbero comunque restare **in attesa** del loro arrivo

---

Esempi di Publish-Subscribe

- **AMQP (Advanced Message Queuing Protocol)**

  Protocollo per comunicazioni di tipo **publish-subscribe** (ma anche **punto-a-punto**)

- **RabbitMQ**

  Middleware di **messaggistica open-source** basato su **AMQP**, ampiamente utilizzato per implementare architetture publish-subscribe

- **MQTT (Message Queuing Telemetry Transport)**

  Protocollo **ISO standard** di **messaggistica leggera**, basato su **TCP/IP**

- **Apache Kafka**

  Piattaforma **open-source** per lo **streaming distribuito di dati**

  - Supporta il modello **publish-subscribe**
  - Utilizzato per **elaborare eventi in tempo reale** e **trasmettere dati** tra applicazioni

---

Esempi di Publish-Subscribe (continua)

- **DDS (Data Distribution Service)**

  Middleware standard basato sul paradigma **publish-subscribe**, usato per la comunicazione **machine-to-machine**

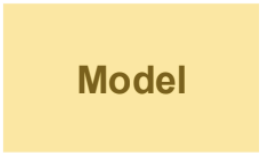
- Mantenuto da **OMG (Object Management Group)**
- Consente comunicazioni **affidabili** tra sensori, controllori e attuatori
- Fornisce un'**API** per serializzazione/deserializzazione di dati (built-in o custom) tramite un **IDL** (Interface Definition Language) dedicato

Esempi d'uso:

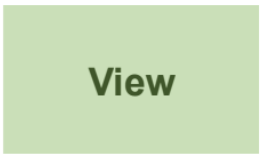
- **NASA**
- **Siemens** (impianti eolici)
- **Volkswagen** e **Bosch** (sistemi di parcheggio autonomo)

Stile Model-View-Controller

Nucleo funzionale
implementa la business logic dell'applicazione
rappresenta i dati su cui opera l'applicazione stessa



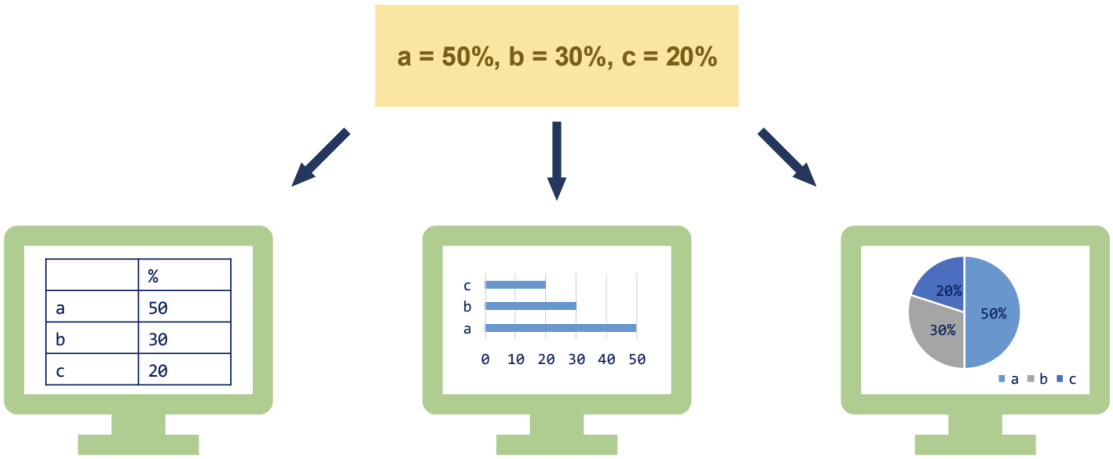
Presentazione del **model** all'utente
ci possono essere più viste per modello



Controllo dell'input dell'utente
Traduce eventi in richieste/operazioni da eseguire su **model**

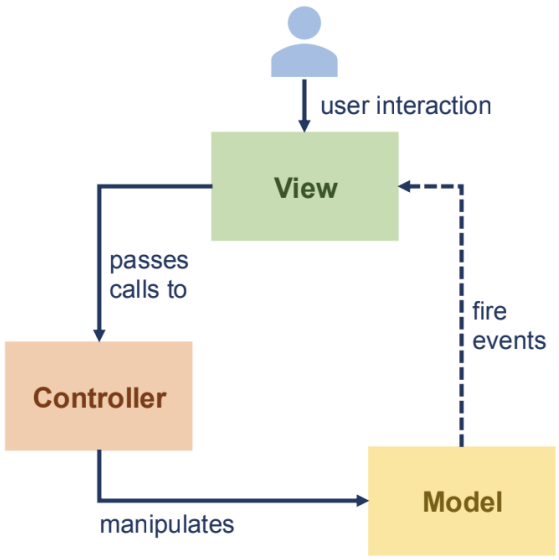


Esempio con model-view diverse:

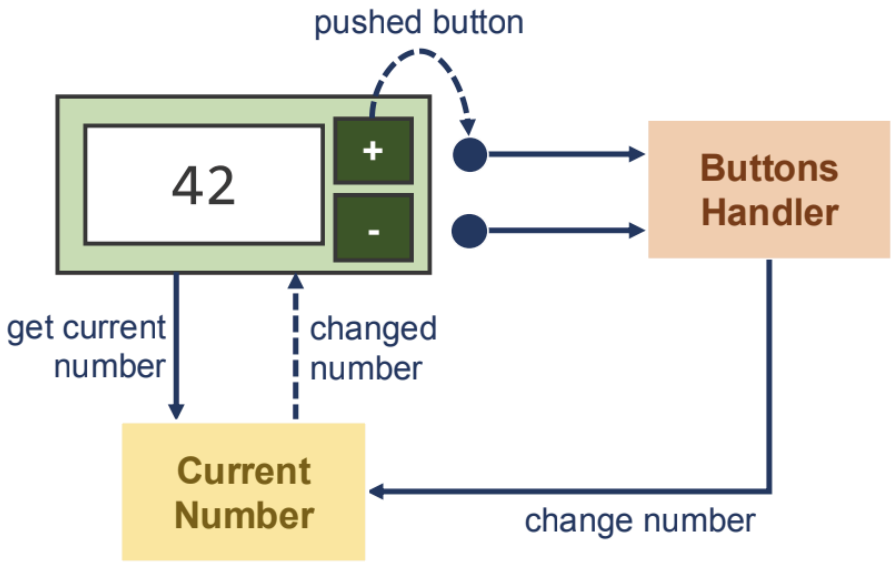


Stile – Model-View-Controller (MVC)

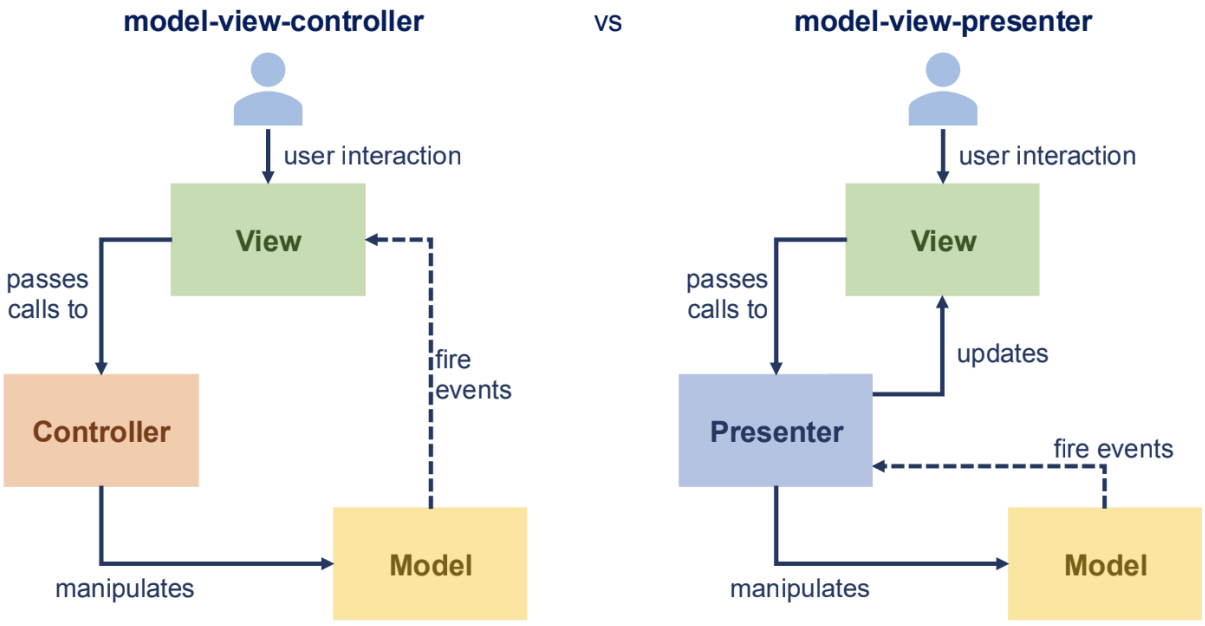
- L'**utente** interagisce con la **view**
- Il **controller** riceve e interpreta le **azioni dell'utente**
- Il **controller** chiede al **model** di **cambiare stato**
- Il **model** notifica la **view** quando **cambia stato**
- La **view** chiede lo **stato** al **model**



Esempio di model-view-controller



Stile Model-View-Presenter



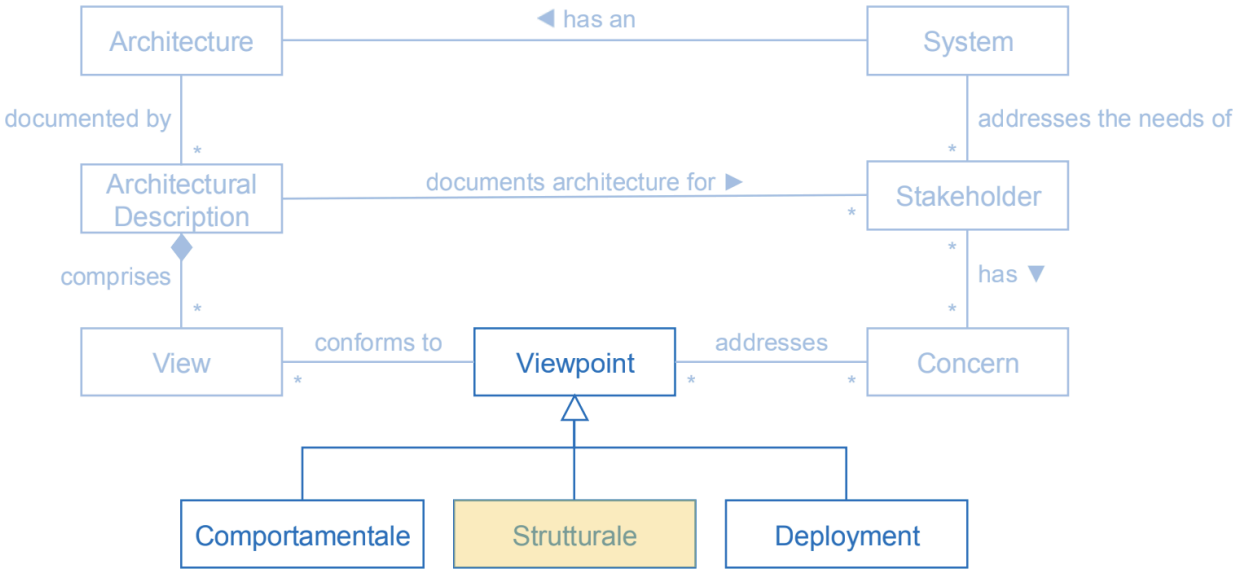
Stile – Process Coordinator

Un componente funziona da **process coordinator (coordinatore)**, mentre gli altri componenti sono **passivi**:

- I componenti (escluso il coordinatore) **non conoscono il loro ruolo** nel processo complessivo
- Ogni componente fornisce un **insieme di funzionalità**

Il **coordinatore** è responsabile della **sequenza di passi** necessari a realizzare un processo:

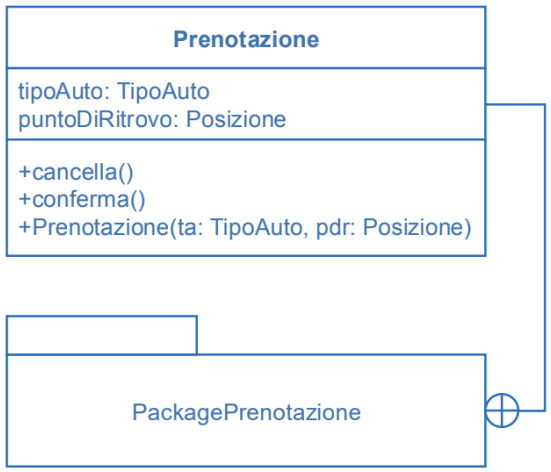
- Riceve la **richiesta**
- Invoca le **funzionalità** offerte dagli altri componenti secondo un **ordine prefissato**
- Fornisce una **risposta**



Vista strutturale

Il **diagramma strutturale** mostra **elementi** e **relazioni**:

- **Elementi:** moduli, ovvero unità di software che realizzano un **insieme coerente di responsabilità** (ad esempio, classi, collezioni di classi, package)
- **Relazioni tra elementi:** del tipo **parte di**, **eredita da**, **dipende da**, **può usare**



Utile per:

- **Costruzione:** schema del codice, directory, file sorgente
- **Analisi:** tracciabilità dei requisiti, impatto di eventuali modifiche
- **Comunicazione:** se gerarchica, offre presentazione top-down per la suddivisione delle responsabilità
- **Progettazione di test:** unità e integrazione

> Non serve per analisi dinamiche, che invece si fanno con viste comportamentali o di deployment

Vista strutturale di decomposizione

Relazione «parte di»

- Una **classe** fa parte di (è contenuta in) un **package**
- Un **package** fa parte di uno più grande

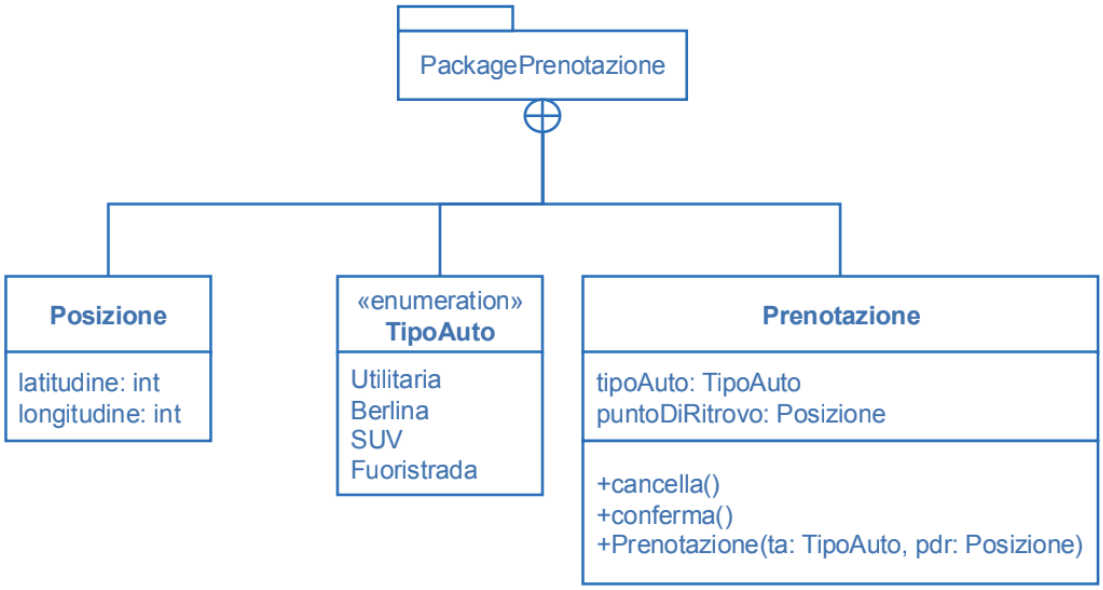
Criteri per raggruppare

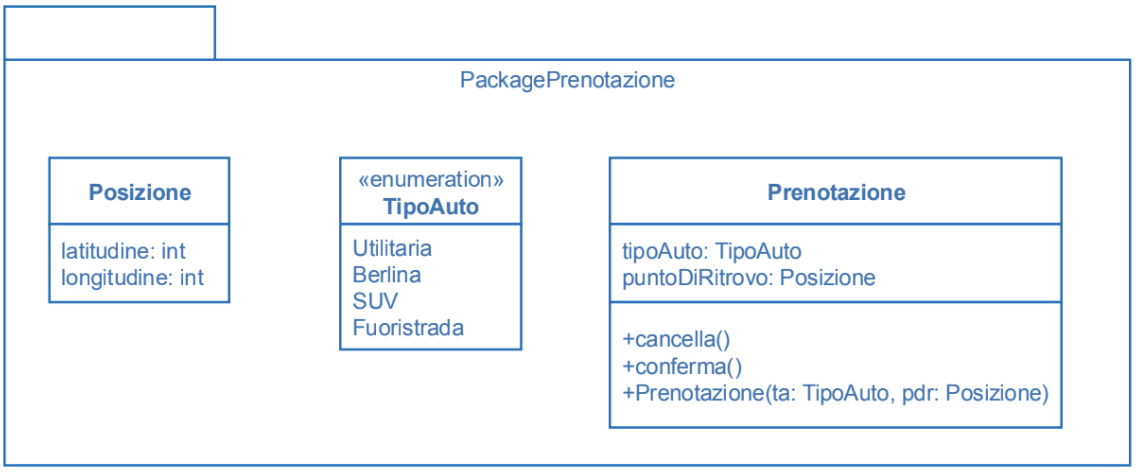
- **Incapsulamento** per facilitare la modificabilità
- Supporto alle scelte **costruisci/compra**
- **Moduli comuni** nelle linee di prodotto

A cosa serve

- **Apprendimento** del sistema
- **Allocazione del lavoro** come punto di partenza

Decomposizione in UML





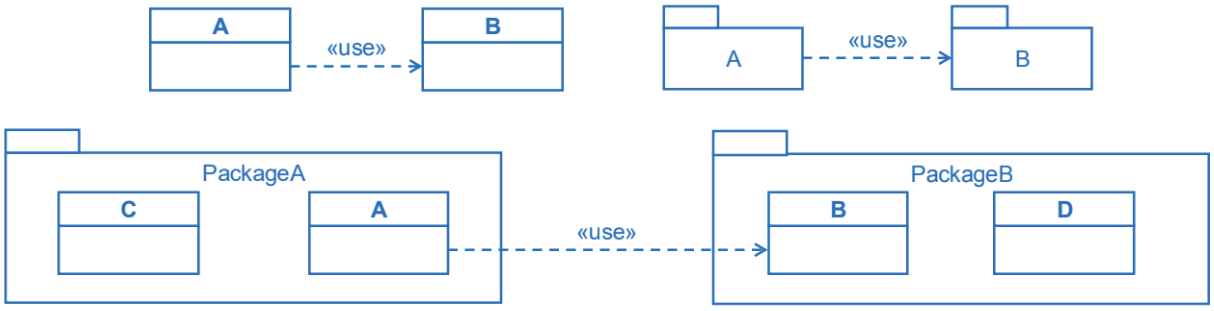
Vista strutturale d'uso

Relazione «use»

- Il **modulo A** usa il **modulo B** se dipende dalla presenza di B per soddisfare i suoi requisiti
- I **cicli** sono permessi, ma **pericolosi**

A cosa serve

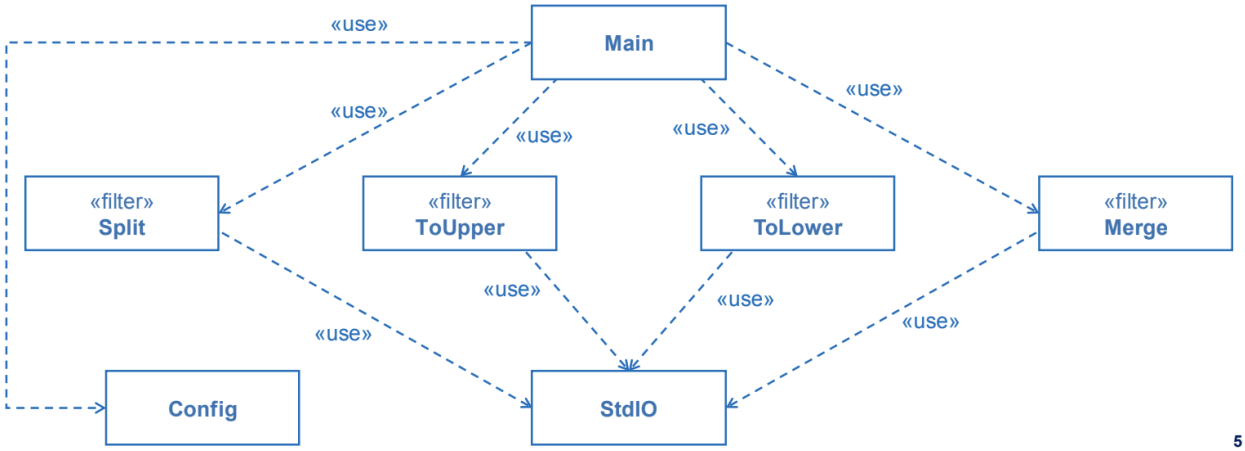
- **Pianificazione** dello sviluppo incrementale
- **Test** di unità e integrazione



Esempio di vista strutturale d'uso

Riprendendo l'esempio di **Pipes & Filters** con biforcazione:

- I **filtri** si chiamano tra loro, ma **non si usano direttamente**
- Il **main** li **configura** per metterli in comunicazione tramite **StdIO** (realizzazione del connettore pipe)
- Tre diversi **"strati"** → possibile interpretazione come **vista a strati**

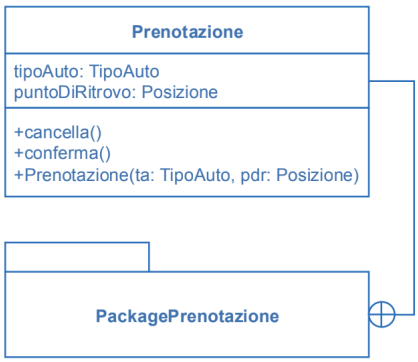


5

Altre viste strutturali: a strati

Elementi

- **Strati:** insieme coeso di moduli (a volte raggruppati in segmenti)
- Ogni strato offre una **interfaccia pubblica** per i suoi servizi



Relazione «allowedToUse»

- Caso particolare di relazione **d'uso**
- **Antisimmetrica** e **non implicitamente transitiva**

A cosa serve

- Favorire **modificabilità** e **portabilità**
- **Controllo della complessità** del sistema

Vista di Deployment

Diagramma

Contiene i seguenti elementi:

- **Artefatti** prodotti da un processo di sviluppo software o dal funzionamento di un sistema (es. codice sorgente, script, file binari, tabelle di database, documenti)
- **Nodi hardware** o, più in generale, **ambienti di esecuzione**

Relazioni tra elementi

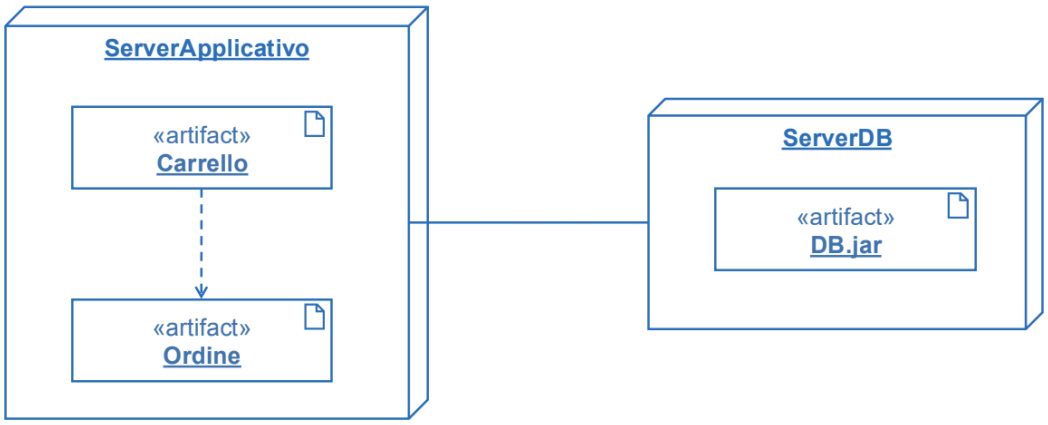
- **Dislocazione** di artefatti negli ambienti di esecuzione
- **Interconnessioni** tra gli ambienti di esecuzione

Utile per

- **Analisi delle prestazioni**
- **Guida per l'installazione** di un sistema

Vista di Deployment in UML

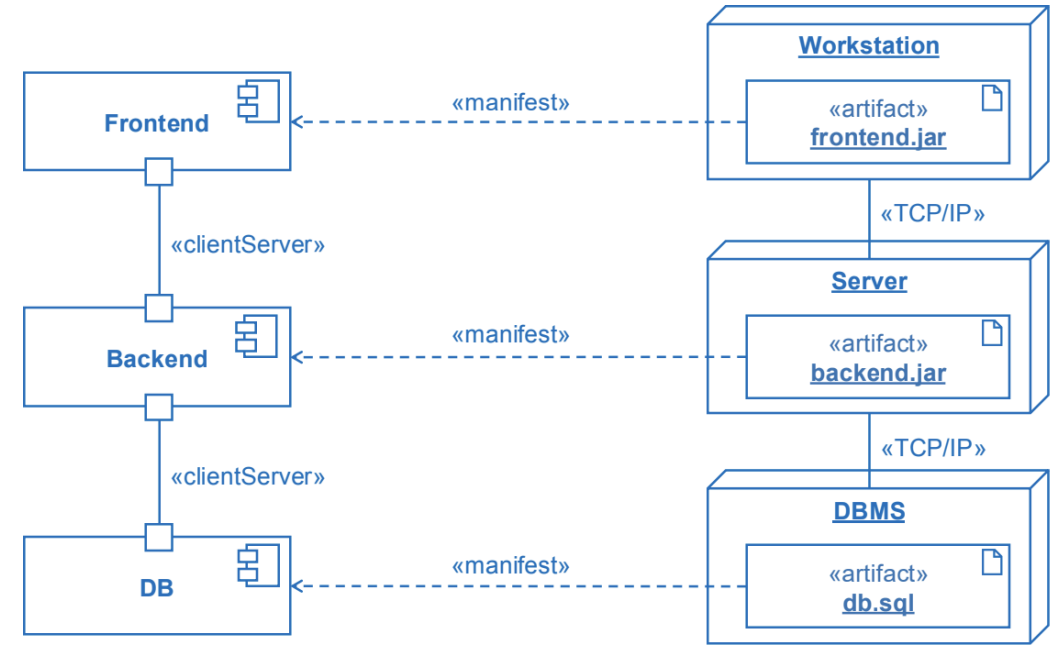
- Gli **ambienti di esecuzione** si rappresentano come **parallelepipedi**
- Gli **artefatti** si rappresentano come elementi con lo **stereotipo «artifact»**
- La **dislocazione di artefatti** negli ambienti di esecuzione si rappresenta con il **contenimento**
- Le **interconnessioni** si rappresentano con **relazioni stereotipate**



Alcune precisazioni

- Si parte spesso dal **deployment di componenti**, mentre in realtà si disloca un **artefatto**
- Un **artefatto "manifesta" un componente**, ovvero ne fornisce l'implementazione
- Un **artefatto** viene **dislocato (deployed)** su un **ambiente di esecuzione**
- L'**installazione** avviene nell'ambiente di esecuzione e comprende **configurazione** e **registrazione** del componente
- L'**istanza di un componente** (a runtime) è quindi **creata a partire dall'artefatto**

Viste Ibride


Workstation
«artifact»
frontend.jar
«manifest»
Frontend
«TCP/IP»
«clientServer»
Server
«artifact»
backend.jar
«manifest»
Backend
«TCP/IP»
«clientServer»
DBMS
«artifact»
db.sql
«manifest»
DB